

Padaria Adonai

Referência do Código

Sistema de Gestão de Padaria

Trabalho Prático 1 · Algoritmos e Estruturas de Dados
ISUTC · Engenharia Informática e de Telecomunicações

Parte 1 · Como funciona o sistema

Parte 2 · Referência técnica: classes, variáveis e funções

Parte 1 · Como funciona a Padaria Adonai

Uma explicação simples, passo a passo

Uma padaria dentro do computador

Imagina uma padaria de brincar que vive dentro do computador. Lá dentro não há pão a sério, mas o computador lembra-se de **todos os pães** que existem: como se chamam, quanto custam, quantos há na prateleira e até quando podem ser comidos. Também se lembra de **tudo o que foi vendido**.

Este programa é o **ajudante** do dono da padaria. Em vez de escrever tudo num caderno, o dono carrega em botões e o computador faz as contas e arruma tudo.

A porta, o porteiro e a loja

Quando abres o programa, passas por três sítios, sempre pela mesma ordem:



1. A porta. Vês uma fotografia de pães com o nome *Padaria Adonai*. Carregas em **Acessar** para entrar.

2. O porteiro. Pergunta quem és e qual é a tua **palavra secreta** (a senha). Se te enganares, ele diz “*Utilizador ou senha inválidos*” e não te deixa passar. Repara que ele não diz *qual* das duas está errada: assim, um espertalhão não consegue adivinhar os nomes dos utilizadores.

3. A loja. À esquerda há uma coluna castanha cheia de botões, um para cada coisa que podes fazer. À direita há duas tabelas: uma com os **pães** (Registos) e outra com as **vendas**.

Em linguagem de programador

Os três sítios são as classes `TelaInicial`, `TelaLogin` e `AplicacaoPadaria`. Nenhuma conhece as outras: quem as liga é o ficheiro `app.py`, como um guia que te leva de sala em sala. Cada ecrã recebe um *callback* (uma função) para chamar quando acaba.

O dono e o ajudante

Na padaria trabalham duas pessoas: o **dono** e o **ajudante** (o funcionário). O dono tem **todas as chaves**: pode pôr pães novos, mudar preços e deitar pães fora. O ajudante pode procurar pães, ver as listas e vender, mas não pode mexer no que há na loja.

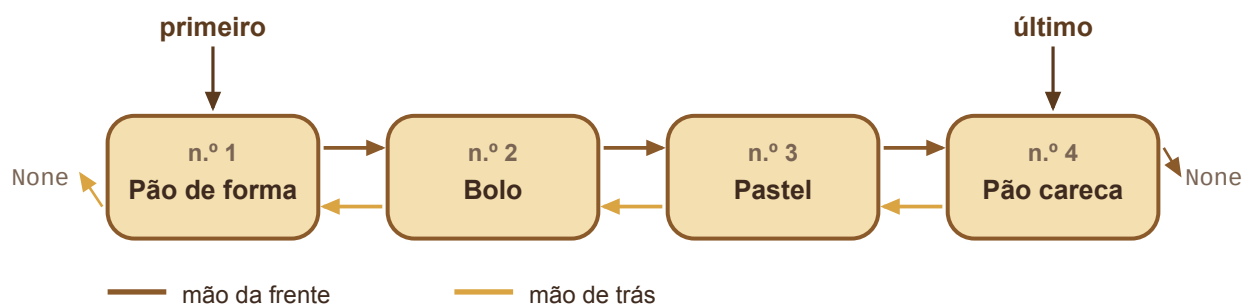
E há um truque: os botões que o ajudante não pode usar **nem sequer aparecem** para ele. É como se essas portas não existissem. E se, por acaso, alguém encontrasse uma dessas portas, há um segundo guarda lá dentro que diz “*Esta operação está reservada ao Dono da Padaria*”.

Em linguagem de programador

A lista das operações proibidas ao ajudante está em `OPERACOES_RESTRITAS_AO_DONO` (`config.py`). O “segundo guarda” é o método `_exige_dono()`.

A fila de pães de mãos dadas

Esta é a parte mais importante. O computador guarda os pães numa **fila**, como meninos numa fila da escola. Cada pão dá **uma mão ao pão da frente** e **outra mão ao pão de trás**.



Cada pão sabe quem está à frente e quem está atrás. As pontas não dão a mão a ninguém (None).

O computador não precisa de se lembrar de onde está cada pão. Só precisa de saber três coisas:

- quem é o **primeiro** da fila;
- quem é o **último** da fila;

- **quantos** pães há.

Para encontrar um pão, começa pelo primeiro e pergunta: “*És tu o número 3?*” Se não for, segue a mão da frente para o próximo e pergunta outra vez, até o encontrar ou até a fila acabar. A isto chama-se **percorrer** a fila.

Em linguagem de programador

A fila é uma **lista duplamente ligada** (`ListaLigada`). Cada pão é um **nó** (`No`) com `proximo` (mão da frente) e `anterior` (mão de trás). A lista guarda `primeiro`, `ultimo` e `tamanho`. Quem “anda” pela fila é a variável `actual`: `actual = actual.proximo`.

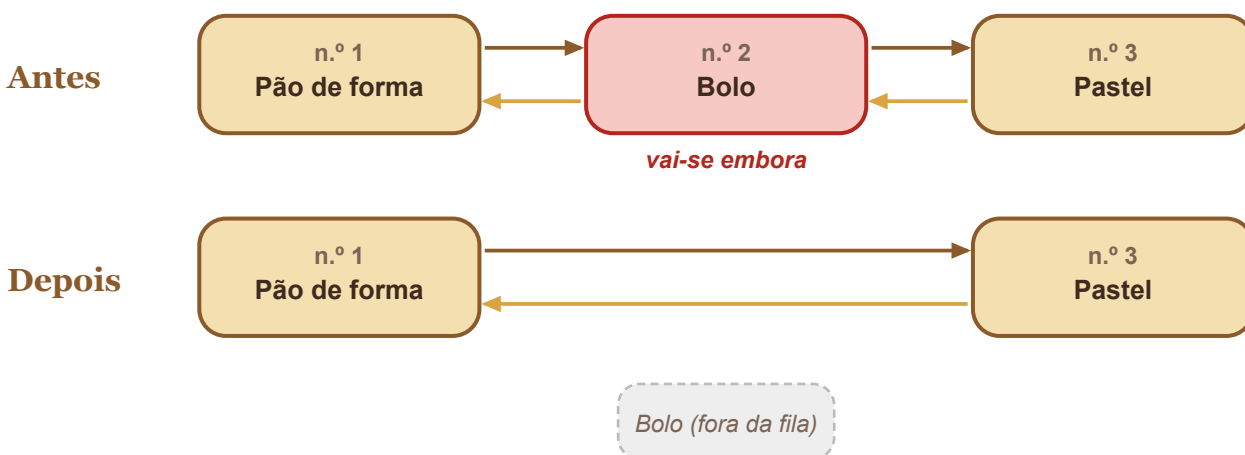
Chega um pão novo

Quando o dono carrega em **Cadastrar**, abre-se uma janela onde escreve os dados do pão novo. O pão novo vai para o **fim da fila**: o último pão dá-lhe a mão, e o pão novo passa a ser o último.

Antes disso, o computador confirma que não há outro pão com o mesmo número, tal como numa equipa de futebol não pode haver dois jogadores com o mesmo número na camisola. Se houver, aparece um aviso, e a janela fica aberta para o dono corrigir sem ter de escrever tudo outra vez.

Um pão vai-se embora

Quando o dono deita um pão fora (**Eliminar**), esse pão sai da fila. Mas a fila não pode ficar partida! Por isso, o pão que estava atrás e o que estava à frente **dão as mãos um ao outro**.



O Bolo sai, e o Pão de forma e o Pastel passam a dar as mãos.

Se quem sai é o primeiro, o segundo passa a ser o primeiro. Se é o último, o penúltimo passa a ser o último.

Também dá para dizer “*tira o 3.º da fila*”. O computador é esperto: se o pão está mais perto do fim, começa a contar pelo último e anda **para trás**, usando a mão de trás. É para isto que servem as duas mãos!

Em linguagem de programador

Remover é sempre feito por `_desligar(no)`, o único sítio que mexe nas ligações para tirar um nó. `eliminar_por_posicao` escolhe começar em primeiro ou em ultimo conforme a posição esteja na primeira ou na segunda metade.

Procurar pães

Há várias maneiras de procurar:

- **Buscar Produto:** pelo número. “*Quero o pão número 3.*”
- **Buscar (1 atributo):** por uma característica. “*Mostra todos os que são Pão.*”
- **Buscar (2 atributos):** por duas ao mesmo tempo. “*Os que são Pão e custam 15.*”

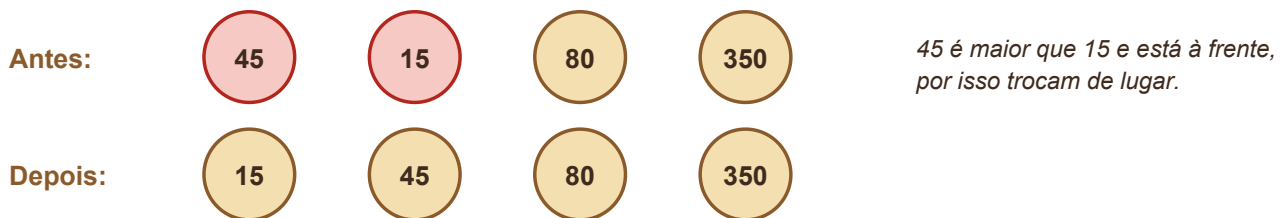
O computador não liga a maiúsculas: “PÃO”, “pão” e “Pão” são a mesma coisa para ele.

Mudar um pão

Se o preço do bolo subir, o dono carrega em **Alterar**, diz o número do bolo, e abre-se a janela **já preenchida**. Ele muda só o que quer e carrega em *Guardar alterações*. É como trocar a etiqueta do preço. O número do pão (o da camisola) **nunca muda**, e por isso esse campo aparece bloqueado.

Pôr os pães por ordem

Para ver os pães do mais barato para o mais caro, o computador usa um método chamado **bolha** (*bubble sort*). Olha para dois pães vizinhos; se o mais caro estiver à frente, **trocamos de lugar**. Faz isto ao longo da fila, muitas vezes, até ninguém precisar de trocar. Os mais caros vão “subindo” para o fim, como bolhas de ar na água.



Um segredo: o computador ordena uma **cópia** da fila, só para mostrar. A fila verdadeira fica igual.

Vender pão

Chega um cliente e pede 3 pães de forma. Carregas em **Registrar Venda**, escreves o número do pão e a quantidade. Antes de vender, o computador confirma três coisas:

1. escreveste números a sério, e a quantidade é maior que zero?
2. esse pão existe?
3. há pães suficientes na prateleira?

Só se as três estiverem certas é que tira os pães da prateleira e escreve a venda no **caderno de vendas**. E o caderno de vendas... é outra fila de mãos dadas! Lá em baixo, o computador soma quantos pães se venderam e quanto dinheiro entrou.

O caderno guarda o **preço daquele dia**. Se amanhã o pão ficar mais caro, a venda de hoje continua com o preço antigo, como num talão de compras.

Em linguagem de programador

As vendas ficam noutra lista duplamente ligada, `ListaVendas`, com nós `NoVenda`. As três verificações estão em `_registar_venda()`, e o stock só é descontado depois de todas passarem.

As cores da padaria

Todas as cores estão guardadas numa só **caixa de lápis**: castanho de crosta, dourado de pão acabado de sair do forno e creme de miolo. Se trocares um lápis dessa caixa, a padaria inteira muda de cor de uma vez.

Em linguagem de programador

A caixa de lápis é o ficheiro `tema.py`, com o dicionário `CORES` e a função `aplicar_tema()`.

Porque é que tudo desaparece quando fecho?

A padaria escreve tudo num **quadro de giz** (a memória do computador), e não num caderno (um ficheiro). Quando fechas o programa, o quadro é apagado. Na próxima vez, começa outra vez com os 4 pães de exemplo e 3 vendas de exemplo.

Onde está cada coisa

O projecto está arrumado em pastas, como as salas de uma casa:

Sala	Ficheiro ou pasta	Para que serve
O interruptor	<code>main.py</code>	Liga o programa.
O guia	<code>src/app.py</code>	Leva-te da porta ao porteiro e do porteiro à loja.
O livro de regras	<code>src/config.py</code>	Nome da padaria, moeda, utilizadores, senhas e quem pode fazer o quê.
O armazém	<code>src/estruturas/</code>	As duas filas de mãos dadas: a dos pães e a das vendas.
A loja	<code>src/interface/</code>	Tudo o que se vê: porta, porteiro, loja e janelas.
A caixa de lápis	<code>src/interface/tema.py</code>	Cores e letras.
A fotografia	<code>assets/imagens/fundo.png</code>	A imagem de pães da porta.
O inspetor	<code>tests/</code>	Verifica que as filas continuam de mãos dadas depois de cada mudança.

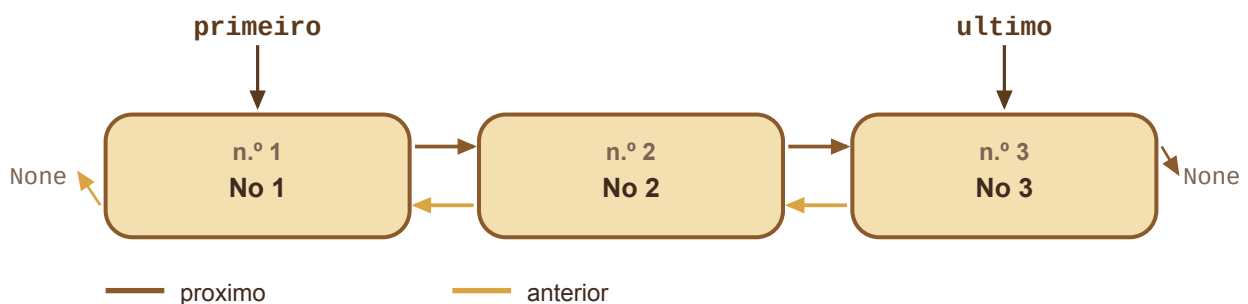
Parte 2 · Referência técnica

Todas as classes, as suas variáveis e o que cada função faz e como

Visão geral

`main.py` chama `main()` em `src/app.py`, que abre três ecrãs por ordem: `TelaInicial` → `TelaLogin` → `AplicacaoPadaria`. Os produtos e as vendas ficam em duas listas duplamente ligadas (`ListaLigada` e `ListaVendas`). A interface lê os dados das janelas, converte os valores, chama um método da lista e volta a desenhar as tabelas. Nada é gravado em disco: cada execução começa com os dados de exemplo.

Regra de dependências: a interface importa as estruturas e a configuração, mas as estruturas nunca importam a interface. É isso que permite testá-las sozinhas.



Cada nó aponta para o anterior e para o próximo. A lista guarda primeiro, ultimo e tamanho.

Invariantes das listas

Verdadeiras antes e depois de qualquer método público:

- lista vazia ↔ primeiro is None ↔ ultimo is None ↔ tamanho == 0
- primeiro.anterior is None e ultimo.proximo is None
- para todo o nó n com próximo: n.proximo.anterior is n

1. Estruturas de dados (src/estruturas)

class No (lista_produtos.py)

Nó da lista de produtos: um produto e as ligações aos vizinhos.

Variável	Tipo	Significado
codigo	int	Código único do produto.
nome	str	Nome do produto.
categoria	str	Categoria (ex.: Pão, Bolo, Doce).
preco	float	Preço unitário (MT).
quantidade	int	Unidades em stock.
validade	str	Data de validade (dd/mm/aaaa).
anterior	No None	Nó anterior; None no primeiro nó.

Variável	Tipo	Significado
<code>proximo</code>	No None	Nó seguinte; None no último nó.

class ListaLigada (lista_produtos.py)

Lista duplamente ligada com todos os produtos.

Variável	Tipo	Significado
<code>primeiro</code>	No None	Primeiro nó (cabeça). None quando a lista está vazia.
<code>ultimo</code>	No None	Último nó (cauda). Permite inserir no fim sem percorrer a lista.
<code>tamanho</code>	int	Número de nós. Atualizado em cada inserção e remoção.

Função	O que faz	Como
<code>__init__()</code>	Cria uma lista vazia.	<code>primeiro = ultimo = None, tamanho = 0.</code>
<code>cadastrar</code> (codigo, nome, categoria, preco, quantidade, validade)	Acrescenta um produto no fim.	Rejeita código repetido (ValueError). Cria um No; se a lista estiver vazia passa a ser primeiro e ultimo; senão liga-o depois do ultimo (<code>novo.anterior = ultimo, ultimo.proximo = novo</code>) e passa a ser o novo ultimo. O(n) por causa da verificação de duplicados; a ligação em si é O(1).
<code>buscar_por_codigo</code> (codigo)	Encontra um produto pelo código.	Percorre a partir de primeiro seguindo proximo; devolve o nó ou None. O(n).
<code>buscar_por_um_atributo</code> (atributo, valor)	Todos os produtos em que um atributo é igual a um valor.	Percorre a lista, lê o atributo com <code>getattr</code> e compara os dois lados como texto sem espaços e em minúsculas ("pão" encontra "Pão"). Devolve uma lista Python de nós.
<code>buscar_por_dois_atributos</code> (a1, v1, a2, v2)	Produtos que cumprem dois atributos ao mesmo tempo.	Mesmo percurso; guarda o nó só se as duas comparações forem verdadeiras (E lógico).
<code>alterar_por_codigo</code> (codigo, novos_dados)	Edita um produto.	Encontra o nó pelo código (ValueError se não existir). Para cada chave/valor do dicionário usa <code>setattr</code> , ignorando valores vazios ("" ou None), que mantêm o valor antigo.
<code>_desligar(no)</code>	Auxiliar interno: retira um nó da cadeia.	Liga os vizinhos entre si nos dois sentidos. Se o nó era o primeiro, primeiro passa para o seguinte; se era o último, ultimo passa para o anterior. Limpa as ligações do nó, decrementa tamanho e devolve o nó. O(1).
<code>eliminar_por_posicao</code> (posicao)	Remove o nó numa posição (1 = primeiro).	IndexError fora de 1..tamanho. Percorre a partir de primeiro (para a frente) se a posição estiver na primeira metade, senão a partir de ultimo (para trás, via anterior); depois chama <code>_desligar</code> .
<code>eliminar_por_codigo</code> (codigo)	Remove o produto com esse código.	ValueError se a lista estiver vazia ou o código não existir; senão <code>buscar_por_codigo + _desligar</code> .
<code>listar_todos()</code>	Todos os produtos pela ordem de cadastro.	Percorre a partir de primeiro e junta os nós numa lista Python.

Função	O que faz	Como
<code>listar_por_critério</code> (atributo, valor)	Produtos que cumprem um critério.	Delega em <code>buscar_por_um_atributo</code> .
<code>listar_ordenado</code> (atributo, decrescente=False)	Produtos ordenados por um atributo.	Copia os nós com <code>listar_todos</code> e aplica bubble sort: passagens repetidas que trocam vizinhos fora de ordem. $O(n^2)$. A lista ligada não é reordenada.

class NoVenda (lista_vendas.py)

Nó da lista de vendas: uma venda. Guarda uma cópia do nome e do preço do momento da venda.

Variável	Tipo	Significado
<code>codigo_produto</code>	int	Código do produto vendido.
<code>nome_produto</code>	str	Nome do produto no momento da venda.
<code>quantidade</code>	int	Unidades vendidas.
<code>preco_unitario</code>	float	Preço por unidade no momento da venda.
<code>total</code>	float	<code>quantidade × preco_unitario</code> , calculado no construtor.
<code>data</code>	str	Data e hora da venda (dd/mm/aaaa hh:mm), de <code>datetime.now()</code> .
<code>anterior</code>	NoVenda None	Venda anterior.
<code>proximo</code>	NoVenda None	Venda seguinte.

class ListaVendas (lista_vendas.py)

Lista duplamente ligada com todas as vendas, pela ordem em que aconteceram. Só se acrescenta: as vendas não se alteram nem se apagam.

Variável	Tipo	Significado
<code>primeiro</code>	NoVenda None	Primeira venda (cabeça).
<code>ultimo</code>	NoVenda None	Última venda (cauda).
<code>tamanho</code>	int	Número de vendas.

Função	O que faz	Como
<code>__init__()</code>	Cria uma lista vazia.	<code>primeiro = ultimo = None, tamanho = 0</code> .
<code>registrar_venda</code> (codigo_produto, nome_produto, quantidade, preco_unitario)	Regista uma venda no fim.	Cria um NoVenda e liga-o depois do <code>ultimo</code> (como em cadastrar). $O(1)$. Devolve o nó novo. Não valida stock (isso é feito na interface).
<code>listar_todas()</code>	Todas as vendas por ordem.	Percorre a partir de <code>primeiro</code> e junta os nós.
<code>total_vendas()</code>	Receita total.	Percorre a lista somando o <code>total</code> de cada nó.
<code>total_quantidade()</code>	Total de unidades vendidas.	Percorre a lista somando a <code>quantidade</code> de cada nó.

2. Interface (src/interface)

Todos os ecrãs usam Tkinter. Cada ecrã recebe a janela principal (root) e um callback que chama quando termina, por isso os ecrãs não precisam de se conhecer.

Módulo tema.py

Cores, fontes e estilos ttk de toda a aplicação. Os outros ficheiros não têm cores escritas à mão.

Variável	Tipo	Significado
CORES	dict	Paleta: fundo, painel, crosta, crosta_escura, dourado, dourado_claro, texto, texto_suave, branco, erro.
FONTE_*	tuple	Fontes: TITULO, SUBTITULO, SECCAO (Georgia) e NORMAL, NEGRITO, PEQUENA (Helvetica).

Função	O que faz	Como
aplicar_tema(root)	Configura todos os estilos.	Ativa o tema ttk clam (o do macOS ignora cores) e define os estilos base e as variantes (Sidebar.TButton, Titulo.TLabel, Topo.TFrame...). Chamar uma vez por cada tk.Tk().
configurar_linhas_alternadas(tabela)	Linhas alternadas numa Treeview.	Regista a tag par com fundo painel.
centrar_janela(janela, largura=None, altura=None)	Centra a janela no ecrã.	Calcula x e y a partir do tamanho do ecrã. Sem tamanho, usa o que o conteúdo pede (wininfo_reqwidth/reqheight).

class TelaInicial (tela_inicial.py)

Ecrã de abertura: imagem de fundo, nome da padaria e botão “Acessar”.

Variável	Tipo	Significado
CAMINHO_IMAGEM_FUNDO	Path (módulo)	Caminho para assets/imagens/fundo.png, calculado a partir deste ficheiro.
LARGURA	int (classe)	Largura da janela: 640 px.
ALTURA_IMAGEM	int (classe)	Altura da zona da imagem: 320 px.
ALTURA_RODAPE	int (classe)	Altura do rodapé com o botão: 90 px.
root	tk.Tk	Janela principal.
ao_acessar	função	Callback chamado ao carregar em “Acessar” (abre o login).
imagem_fundo	tk.PhotoImage	A imagem de fundo. Guardada como atributo para o Python não a apagar da memória.

Função	O que faz	Como
__init__(root, ao_acessar)	Constrói o ecrã.	Título, tamanho e centragem; desenha a imagem e os textos num Canvas; rodapé escuro com o botão; Enter também entra.

Função	O que faz	Como
<code>_texto_com_sombra</code> (<code>canvas</code> , <code>x</code> , <code>y</code> , <code>texto</code> , <code>fonte</code>)	Texto legível sobre a imagem.	Desenha o texto duas vezes: uma sombra escura deslocada 2 px e o texto claro por cima.
<code>_acessar()</code>	Sai do ecrã.	Remove o atalho do Enter, destrói os widgets e chama <code>ao_acessar()</code> .

class TelaLogin (tela_login.py)

Ecrã de login. Só passa quem estiver em UTILIZADORES.

Variável	Tipo	Significado
LARGURA / ALTURA	int (classe)	Tamanho da janela: 420 × 480 px.
root	tk.Tk	Janela principal.
ao_autenticar	função	Callback chamado com o registo do utilizador depois de um login certo.
entrada_utilizador	ttk.Entry	Campo do utilizador.
entrada_senha	ttk.Entry	Campo da senha (mostra •).
label_erro	ttk.Label	Mostra “Utilizador ou senha inválidos.”

Função	O que faz	Como
<code>__init__</code> (<code>root</code> , <code>ao_autenticar</code>)	Constrói o formulário de login.	Logo, nome da padaria, dois campos, etiqueta de erro e botão “Entrar”. Enter no utilizador salta para a senha; Enter na senha tenta entrar.
<code>_tentar_login()</code>	Verifica as credenciais.	Procura o utilizador em UTILIZADORES e compara a senha. Errado: mostra o erro e limpa a senha. Certo: destrói os widgets e chama <code>ao_autenticar(registo)</code> .

class AplicacaoPadaria (aplicacao.py)

Janela principal depois do login: barra de topo, sidebar com as operações e duas abas (Registos / Vendas). Os dados do produto são pedidos numa janela própria, e não num formulário fixo.

Variável	Tipo	Significado
ATRIBUTOS	list (classe)	Campos do produto por ordem: código, nome, categoria, preço, quantidade, validade.
ATRIBUTOS_LABEL	dict (classe)	Nome amigável de cada campo (ex.: preço → “Preço”).
ATRIBUTOS_DICA	dict (classe)	Dica ao lado do campo na janela de produto (preço → MT, validade → dd/mm/aaaa).
LARGURA, ALTURA	int (classe)	Tamanho inicial: 1150 × 700 px (mínimo 950 × 600).
lista	ListaLigada	Todos os produtos.
lista_vendas	ListaVendas	Todas as vendas.
root	tk.Tk	Janela principal.
perfil	dict	Registo do utilizador autenticado (senha, tipo, nome).

Variável	Tipo	Significado
e_dono	bool	True se o perfil for PERFIL_DONO.
ao_terminar_sessao	função	Callback chamado ao sair (recomeça na tela inicial).
container	tk.Frame	Contém a sidebar (esquerda) e a área das abas (direita).
abas	tk.Notebook	O controlo das abas.
tab_registos / tab_vendas	tk.Frame	As duas abas.
tabela_registos	tk.Treeview	Tabela de produtos (uma coluna por atributo).
tabela_vendas	tk.Treeview	Tabela de vendas.
frame_resumo_vendas	tk.Frame	Barra por baixo da tabela de vendas com os totais e um botão.
label_total_qtd / label_total_vendas	tk.Label	Total de unidades vendidas e receita total (MT).

Construção do ecrã

Função	O que faz	Como
<code>__init__</code> (root, perfil, ao_terminar_sessao)	Inicia a janela principal.	Cria as duas listas, guarda o perfil, calcula e_dono, define título e tamanho, centra, carrega os dados de exemplo e constrói o layout.
<code>_construir_layout()</code>	Constrói a janela toda.	Barra de topo; depois o container com a sidebar e as abas.
<code>_construir_barra_topo()</code>	Barra de topo.	Nome da padaria, saudação, perfil e botão “Sair”.
<code>_saudacao()</code>	Texto de boas-vindas.	Bom dia / Boa tarde / Boa noite conforme a hora, com o nome do utilizador.
<code>_terminar_sessao()</code>	Sair.	Pede confirmação, destrói a janela e chama <code>ao_terminar_sessao</code> .
<code>_construir_sidebar()</code>	Botões das operações.	Percorre os pares (texto, método) e cria um botão para cada um. Salta as operações em <code>OPERACOES_RESTRITAS_AO_DONO</code> quando o utilizador não é o dono.
<code>_construir_abas(parent)</code>	As duas abas.	Registos: Treeview com uma coluna por atributo e scrollbar. Vendas: Treeview de vendas, totais e botão “+ Registar Venda”. Preenche as duas tabelas.
<code>_popular_exemplo()</code>	Dados de exemplo.	Acrescenta 4 produtos e 3 vendas.

Auxiliares

Função	O que faz	Como
<code>_atualizar_tabela_registos (produtos=None)</code>	Redesenha a tabela de produtos.	Limpa a Treeview e insere os produtos dados, ou todos se não for dado nenhum. Linhas alternadas.
<code>_atualizar_tabela_vendas ()</code>	Redesenha a tabela de vendas.	Limpa e volta a preencher a partir de <code>lista_vendas</code> ; atualiza os dois totais.
<code>_exige_dono()</code>	Verificação de permissão.	Se o utilizador não for o dono, mostra “Acesso restrito” e devolve True (quem chamou pára).
<code>_nova_janela(titulo)</code>	Cria um diálogo.	Toplevel por cima da janela principal (transient), sem redimensionar; devolve (janela, corpo).
<code>_mostrar_janela(janela)</code>	Mostra o diálogo.	Centra-o e torna-o modal (<code>grab_set</code>).
<code>_janela_produto (titulo, texto_botao, ao_confirmar, produto=None)</code>	Janela com os dados do produto (Cadastrar e Alterar).	Um campo por atributo, com dicas. Com produto, vem preenchida e com o código bloqueado. Ao confirmar chama <code>ao_confirmar(dados, janela)</code> ; se devolver False a janela fica aberta.
<code>_combo_atributos(parent)</code>	Combobox de atributos.	Mostra os nomes amigáveis (“Preço”).
<code>_atributo_escolhido (combo)</code>	Lê a Combobox.	Converte a escolha no nome interno (“preco”), ou None se nada foi escolhido.
<code>_pedir_atributo_valor (titulo)</code>	Diálogo: escolher atributo e escrever valor.	Modal (<code>wait_window</code>); avisa se não houver atributo; devolve (atributo, valor) ou (None, None).
<code>_pedir_valor_simples (titulo, label_texto)</code>	Diálogo: escrever um valor.	Mesmo padrão com um só campo; devolve o texto ou None.

Ações (botões da sidebar)

Função	O que faz	Como
<code>acao_buscar_produto()</code>	Busca por código.	Diálogo pede o código; <code>buscar_por_codigo</code> ; mostra só esse produto e seleciona a linha.
<code>acao_cadastrar()</code> [dono]	Novo produto.	Abre <code>_janela_produto</code> vazia. Converte código/quantidade para int e preço para float, chama <code>lista.cadastrar</code> e atualiza a tabela. Números inválidos ou código repetido mostram erro e mantêm a janela aberta.
<code>acao_alterar()</code> [dono]	Editar produto.	Pede o código, abre <code>_janela_produto</code> preenchida; campos vazios mantêm o valor; chama <code>alterar_por_codigo</code> .
<code>acao_buscar_um_atributo()</code>	Busca por 1 atributo.	<code>_pedir_atributo_valor + buscar_por_um_atributo</code> ; mostra os resultados ou “Nenhum produto encontrado”.
<code>acao_buscar_dois_atributos()</code>	Busca por 2 atributos.	Diálogo com dois pares atributo/valor; <code>buscar_por_dois_atributos</code> .
<code>acao_eliminar_posicao()</code> [dono]	Eliminar por posição.	<code>_pedir_valor_simples</code> ; <code>eliminar_por_posicao</code> ; mostra o nome do produto removido.
<code>acao_eliminar_codigo()</code> [dono]	Eliminar por código.	<code>_pedir_valor_simples</code> ; <code>eliminar_por_codigo</code> .
<code>acao_listar_todos()</code>	Mostrar todos.	Atualiza a tabela e seleciona a aba Registos.
<code>acao_listar_criterio()</code>	Listar por critério.	<code>_pedir_atributo_valor + listar_por_criterio</code> .
<code>acao_listar_ordenado()</code>	Listar ordenado.	Diálogo com Combobox de atributo e caixa “Ordem decrescente”; <code>listar_ordenado</code> .
<code>_registar_venda()</code>	Registar venda (sidebar e aba Vendas).	Pede código e quantidade. Verifica: inteiros positivos, produto existe, stock suficiente. Só depois desconta o stock, chama <code>registar_venda</code> , atualiza as duas tabelas e mostra o total.

3. Configuração e arranque

config.py

Variável	Tipo	Significado
<code>NOME_PADARIA</code>	str	“Padaria Adonai”. Usado nos títulos e ecrãs.
<code>MOEDA</code>	str	“MT”. Usado em todos os valores monetários.
<code>PERFIL_DONO /</code> <code>PERFIL_FUNCIONARIO</code>	str	Nomes dos perfis; compara-se sempre com estas constantes.
<code>UTILIZADORES</code>	dict	<code>utilizador → {senha, tipo, nome}</code> . Dois: <code>dono / dono123</code> (Dono da Padaria) e <code>funcionario / func123</code> (Funcionário).
<code>OPERACOES_RESTRICTAS_AO_DONO</code>	set	Texto dos botões que só o dono vê: Cadastrar, Alterar (por código), Eliminar por posição, Eliminar por código.

app.py e main.py

Função	O que faz	Como
<code>main()</code>	Início do programa.	Cria a janela com <code>criar_janela_raiz</code> , chama <code>iniciar_aplicacao</code> e arranca <code>mainloop()</code> .
<code>criar_janela_raiz()</code>	Cria uma janela Tk com o tema.	<code>tk.Tk()</code> + <code>aplicar_tema</code> .
<code>iniciar_aplicacao(root)</code>	Encadeia os três ecrãs.	Define dois callbacks: <code>ao_acessar</code> abre <code>TelaLogin</code> , <code>ao_autenticar(perfil)</code> abre <code>AplicacaoPadaria</code> . Depois mostra <code>TelaInicial</code> .
<code>ao_terminar_sessao()</code>	Recomeça depois de sair.	Cria uma janela nova e chama <code>iniciar_aplicacao</code> outra vez.
<code>main.py</code>	Ponto de entrada.	<code>python main.py</code> → <code>src.app.main()</code> .

4. Limites conhecidos

- **Ordenar por validade** compara texto, por isso “01/12/2026” fica antes de “15/09/2026”. Solução: guardar a validade como `datetime.date`.
- **Os dados perdem-se** ao sair ou fechar. Solução: gravar num ficheiro (JSON/SQLite) e carregar em `AplicacaoPadaria.__init__`.
- **Buscas por código são $O(n)$** . Solução para muitos produtos: um dicionário `{codigo: No}` ao lado da lista.
- **Senhas em texto simples** no código: aceitável só num trabalho académico.